

Deployment Discussions - Sanity Checks to Verify Expected Behavior and Alignment with Best Practices

Purpose

This note captures the deployment decisions that were checked before and during production rollout. It records the expected behavior and the reasoning behind the narrow deployment path; it is not a substitute for the implementation or architecture documentation.

App Engine boundary

- Use Google App Engine Standard, default service, Python 3.11.
- Keep the low-traffic configuration deliberately small: F1, automatic scaling, minimum zero and maximum one instance.
- The active Google Cloud project is selected by local gcloud configuration. The project ID is not hard-coded into application source or app.yaml.
- `GOOGLE_CLOUD_PROJECT` is supplied by the Google runtime when the application needs the project identity.

Secrets and environment separation

- Local development continues to use the ignored project-root `.env` file and the development ActivityInfo database.
- Production uses a separate ActivityInfo database.
- `ACTIVITYINFO_DATABASE_ID` is environment-specific configuration, not a secret.
- `ACTIVITYINFO_API_TOKEN` is a secret and is stored in Google Secret Manager under `activityinfo-api-token`.
- The App Engine runtime service account has Secret Manager Secret Accessor on that secret. The user's own Google account already has project-owner access and does not need a redundant secret-level grant.
- Secret loading is lazy so imports, offline tests, and `/health` do not require Google credentials or touch Secret Manager.

ActivityInfo bootstrap and publication

- ActivityInfo databases are created manually. Bootstrap creates and verifies the Web Monitor resources inside the configured database.
- Form IDs are deterministic functions of the database ID. Persistence therefore does not need a separate registry for form IDs.
- Report IDs follow the same deterministic strategy.
- Published report URLs are treated differently: the URL shape is provider-owned presentation metadata and is intentionally not inferred from report IDs.
- Production reporting therefore uses two passes: create/publish the deterministic Single and Notebook reports, then copy the exact ActivityInfo-generated Single embed URL and Notebook standalone URL into production configuration.

Cloud Scheduler and retries

- Cloud Scheduler uses an App Engine target rather than legacy `cron.yaml`.
- The scheduled endpoint is `POST /tasks/monitor` and is protected by App Engine's admin handler boundary.
- Scheduler headers identify the logical work but are not treated as cryptographic authentication by themselves.
- A logical scheduled invocation is identified by job identity plus `X-CloudScheduler-ScheduleTime`. The scheduled time remains stable across retry attempts.
- Scheduled Crawl and potential Snapshot IDs are deterministic for a site and invocation.

- A retry of the same invocation reuses an already-completed Crawl rather than crawling or writing again.
- If some sites finished before an infrastructure failure, a retry skips the completed sites and continues the remainder where the persisted state can be verified safely.
- ActivityInfo does not provide a transactional distributed lock here, so the design documents the remaining rare cross-instance race instead of claiming mathematical exactly-once execution.

Production sanity checks

- Verify the intended gcloud account and project before every cloud mutation or deploy.
- Verify the production bootstrap can be rerun without duplicate forms, seeds, or reports.
- Verify Secret Manager access without logging the token.
- Verify /health, the main UI, history, and /reports against production data.
- Use a genuine scheduled delivery to test X-CloudScheduler-ScheduleTime behavior; a manual Scheduler run was observed to omit the required timestamp.
- Replay the same recorded job/time with crawling and writes forbidden to prove the deduplication path.
- Restore the daily 00:00 UTC schedule after the near-term acceptance run.
- Keep only the default F1 App Engine service/version and avoid adding cloud products that the application does not require.

Observed acceptance

The first genuine scheduled production delivery recorded four Crawls: three initial baselines and one IMDb error. It created three Snapshots. The application history and ActivityInfo reports reflected those records, and a replay of the same invocation reused all four Crawls without crawling or writing.